

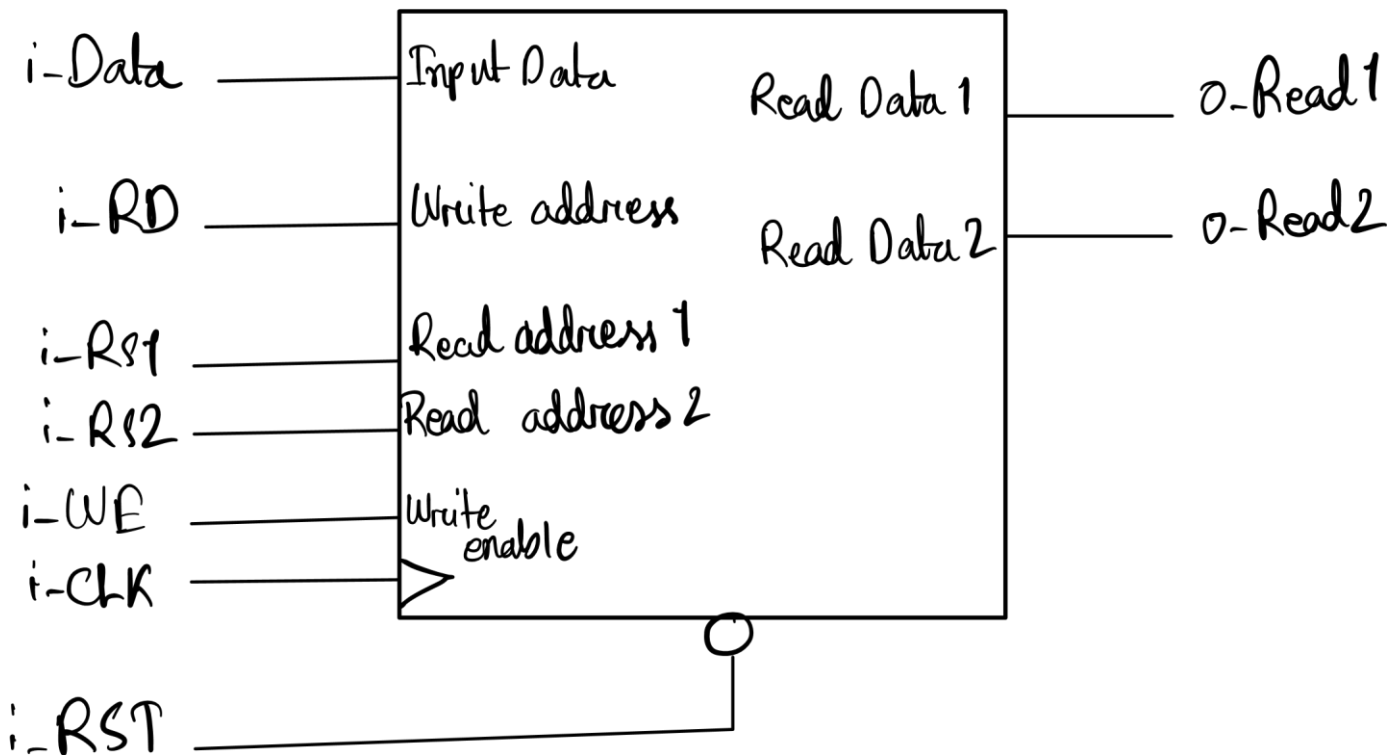
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

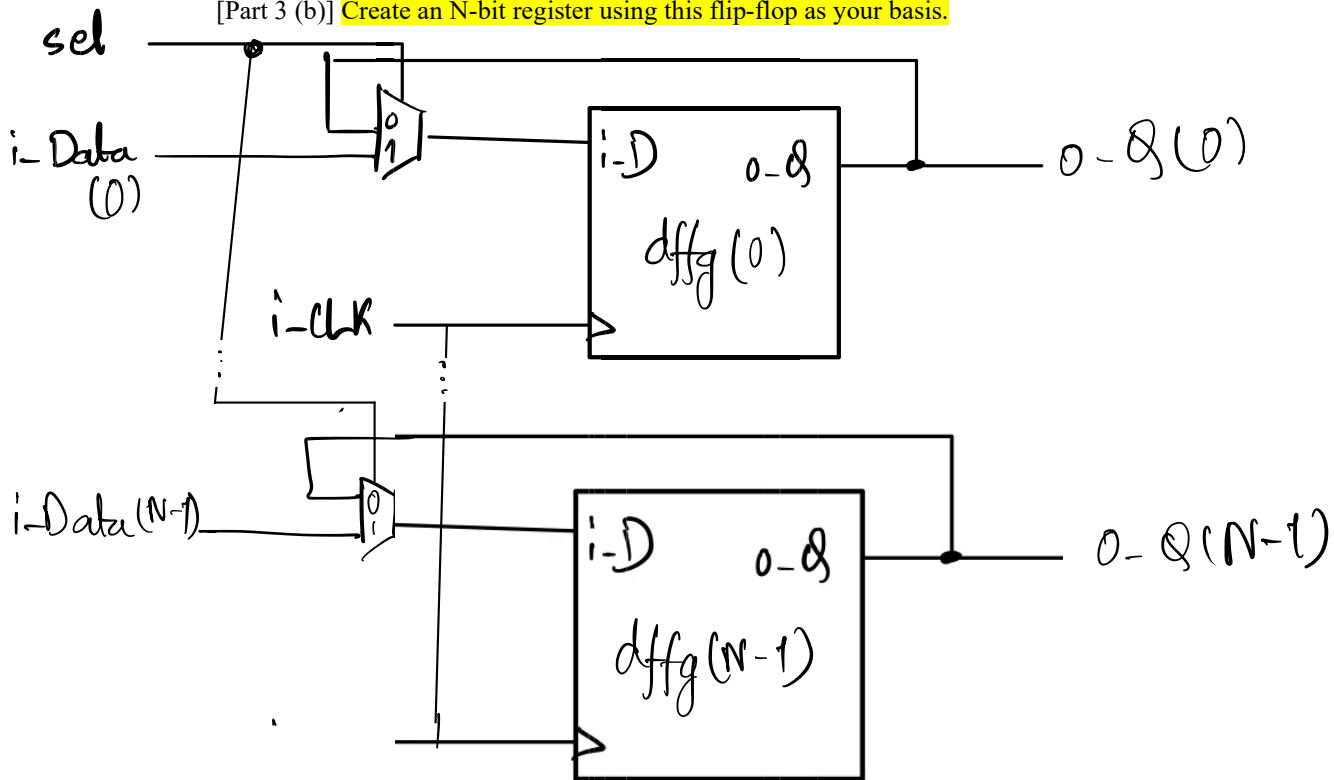
Student Name Sayon Saha

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?

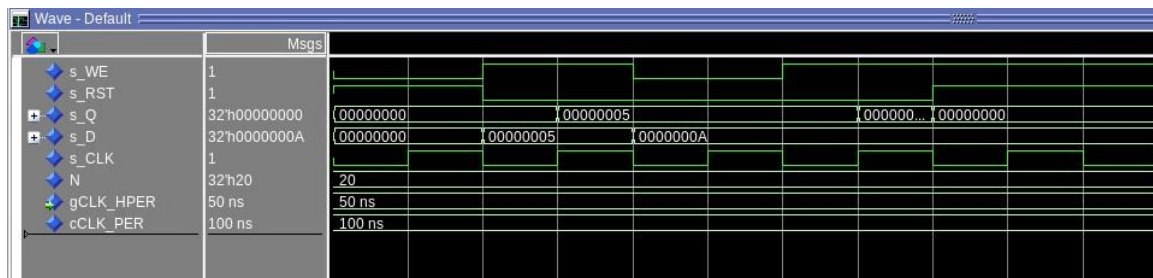


[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.



[Part 3 (c)] Waveform.

The following is the waveform from the testbed simulation of an n-bit register:



Register waveform signal key

s_D is the input word, s_Q is the stored output, s_WE enables writing, s_RST clears the register, and s_CLK supplies the write edge. $N = 0x20$ represents 32 bits; $CLK_PER = 100$ ns and $gCLK_HPER = 50$ ns describe the clock timing.

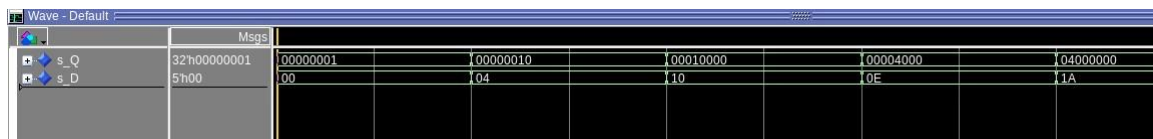
The register waveform demonstrates reset, enabled writes, and holding a value. Initially $s_RST = 1$ clears s_Q to 00000000. After reset is released and $s_WE = 1$, the rising clock edge captures $s_D = 00000005$. When s_D changes to 0000000A while $s_WE = 0$, s_Q stays at 00000005; it updates to 0000000A only after writing is enabled and a rising edge occurs. Asserting s_RST again clears s_Q to zero asynchronously, without waiting for the next clock edge. All data values in this figure are hexadecimal.

[Part 3 (d)] What type of decoder would be required by the RISC-V register file and why?

A 5-to-32 decoder would be required. This is because there are 32 registers and a 5-bit address can identify each register uniquely. The decoder converts the input 5bit address value into one-hot output values. Combined with the write-enable signal, this selects the destination register.

[Part 3 (e)] Waveform.

The following is a waveform simulation of the 5-to-32 decoder:



The decoder maps the 5-bit input s_D to a one-hot 32-bit output s_Q . The visible hexadecimal input/output pairs are 00 -> 00000001, 04 -> 00000010, 10 -> 00010000, 0E -> 00004000, and 1A -> 04000000. Each output has only the bit corresponding to the selected register number set. These values are selection masks, not the contents stored in the registers. The output changes combinatorially; no clock is required.

[Part 3 (f)] In your write-up, describe and defend the design you intend on implementing for the next part.

I intend to implement the 32-bit 32-to-1 multiplexer using dataflow VHDL with a with select statement. The inputs will be organized as an array of 32 words, each containing 32 bits. A 5-bit selection signal will choose which word is connected to the output, since $(2^5 = 32)$. This approach directly expresses the required selection behavior and is easier to read and verify than manually connecting multiple smaller multiplexers. The circuit is combinational, so the output responds to changes in the selected input or selection signal without requiring a clock.

[Part 3 (g)] Waveform.

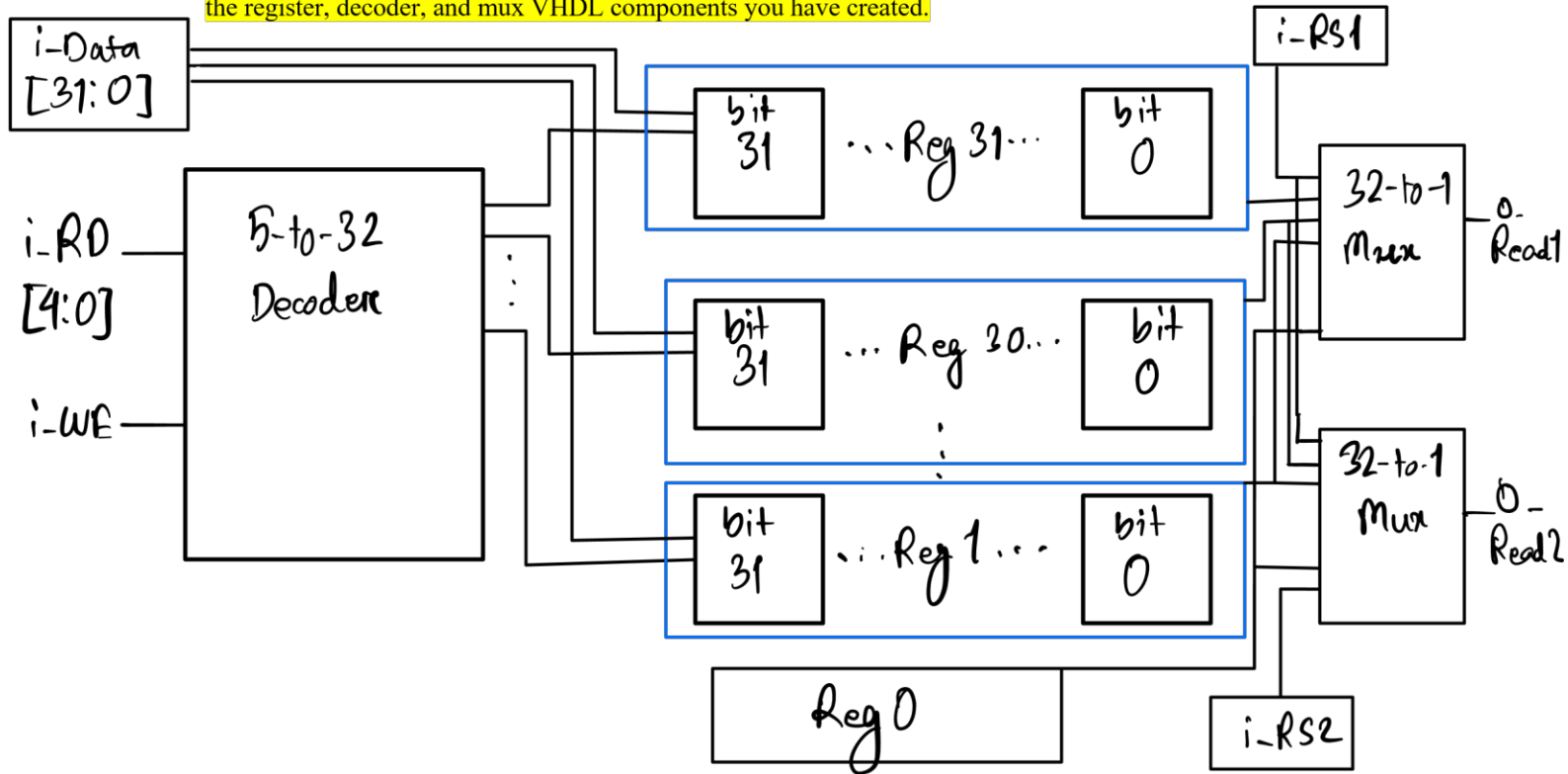
The following is the waveform simulation for the 32 bit 32to1 mux:



Multiplexer waveform annotation

s_sel selects one word from the s_data array, and s_out carries that complete 32-bit word. The visible selections 00, 05, and 1F correspond to inputs 0, 5, and 31. Their outputs are 11111101, 5F5F5F5F, and 10101010, respectively, in hexadecimal. Returning s_sel to 00 restores 11111101. The unchanged input array and changing output demonstrate combinational selection without a clock.

[Part 3 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.



[Part 3 (i)] Waveform.

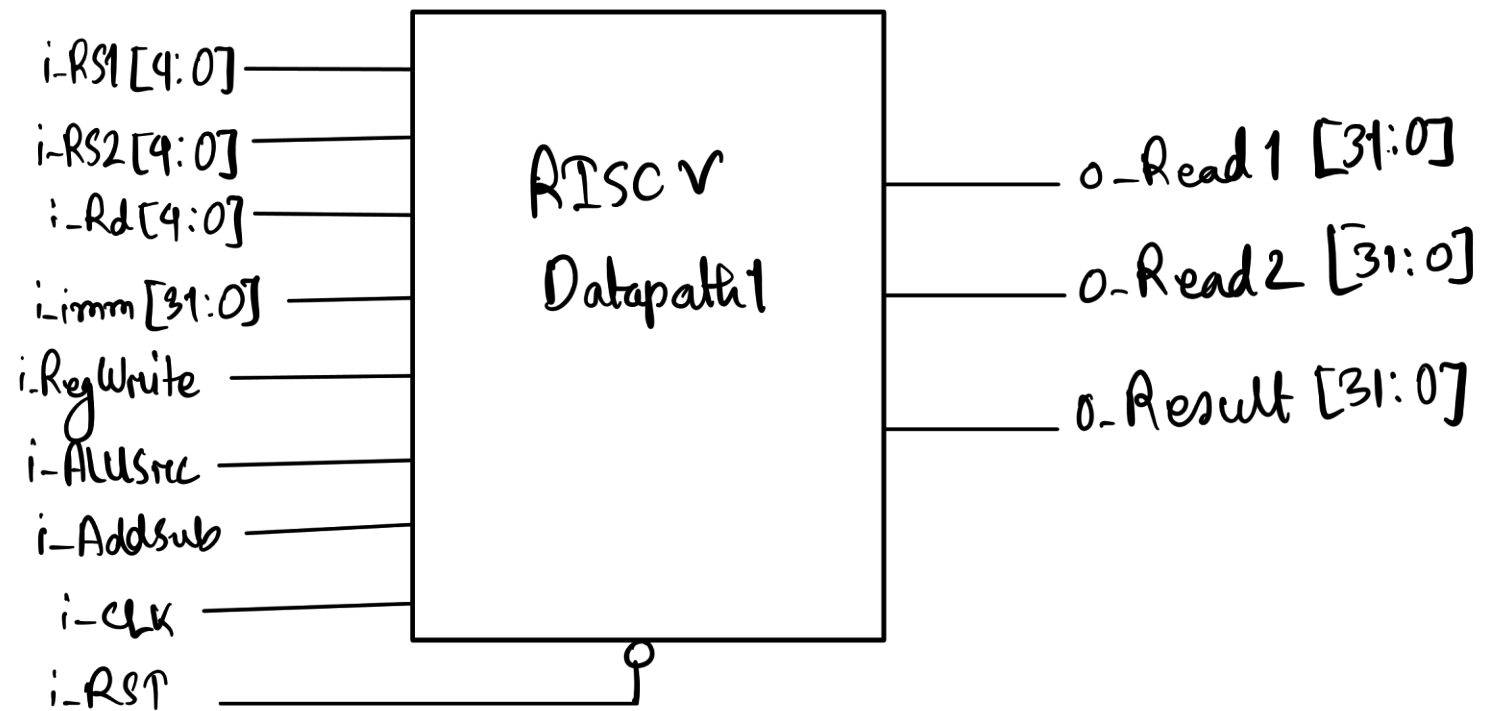


Register file waveform annotation

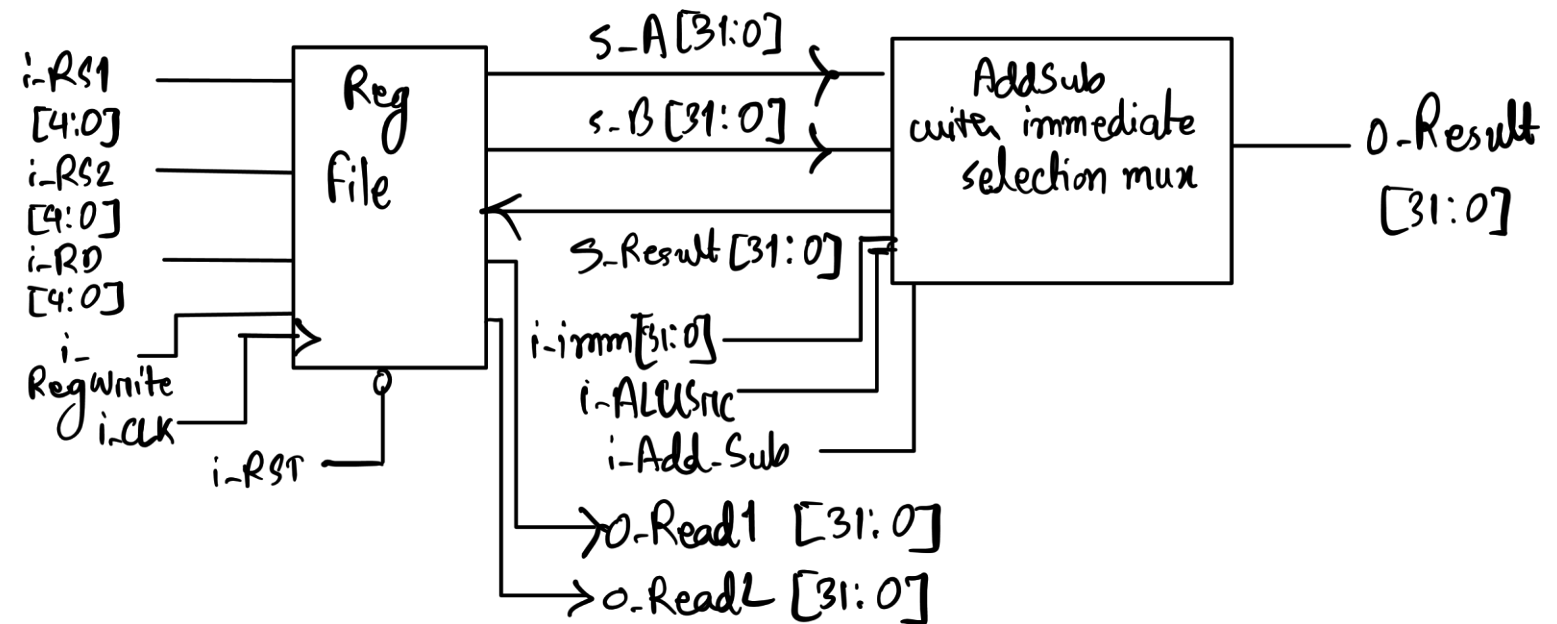
The waveform at the bottom of the schematic uses decimal values. After reset, $s_WE = 1$ allows 10 to be written to register 5 and then 20 to register 10 at rising clock edges. s_RS1 and s_RS2 select the read registers; s_Read1 and s_Read2 return their stored values independently.

When $s_WE = 0$, the attempted write of -1 to register 5 is ignored, so reading it still returns 10. The final attempted write to register 0 is also ignored: reading x0 returns zero while reading x10 still returns 20. This verifies write-enable behavior, two independent read ports, and protection of x0.

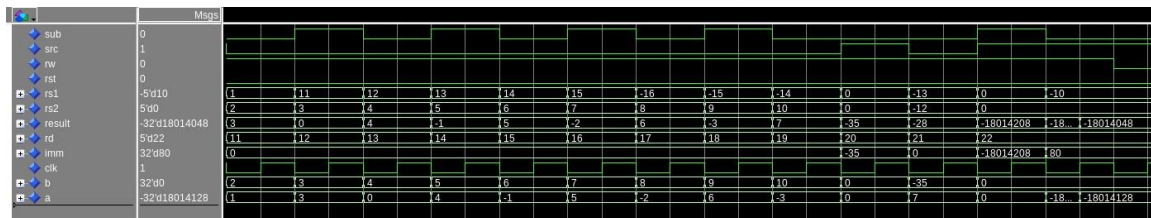
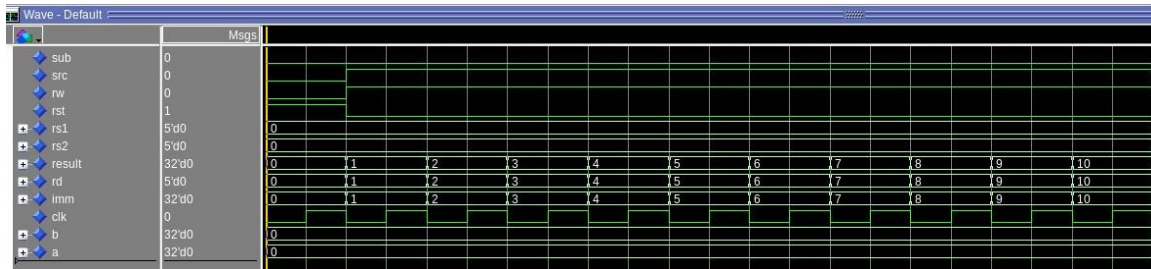
[Part 4 (b)] Draw a symbol for this RISC-V-like datapath.



[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final state of the register file should be and provide a mapping of the registers used in your drawing to the register names.



Registers			
Floating Point		Control and Status	
Name	Number	Value	
zero	0	0x00000000	
ra	1	0x00000001	
sp	2	0x00000002	
gp	3	0x00000003	
tp	4	0x00000004	
t0	5	0x00000005	
t1	6	0x00000006	
t2	7	0x00000007	
s0	8	0x00000008	
s1	9	0x00000009	
a0	10	0x0000000a	
a1	11	0x00000003	
a2	12	0x00000000	
a3	13	0x00000004	
a4	14	0xffffffff	
a5	15	0x00000005	
a6	16	0xfffffffffe	
a7	17	0x00000006	
s2	18	0xfffffffffd	
s3	19	0x00000007	
s4	20	0xffffffffdd	
s5	21	0xffffffe4	
s6	22	0xfeed2050	
s7	23	0x00000000	
s8	24	0x00000000	
s9	25	0x00000000	
s10	26	0x00000000	
s11	27	0x00000000	
t3	28	0x00000000	
t4	29	0x00000000	
t5	30	0x00000000	
t6	31	0x00000000	
pc		0x004000e0	

[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

Simple datapath waveform annotations

First waveform: rst initially clears the registers. With sub = 0, src = 1, and rw = 1, the datapath adds each immediate to x0 and writes values 1 through 10 into x1 through x10. rs1 and rs2 identify source registers, a and b carry their values, rd selects the destination, and result is the ALU output fed back to the register file.

Second waveform: src = 0 selects register arithmetic, and sub alternates addition and subtraction. The values written into x11 through x19 are 3, 0, 4, -1, 5, -2, 6, -3, and 7. The remaining operations produce x20 = -35, x21 = -28, and x22 = 0xFEED2050 (-18014128). The negative-looking 5-bit source addresses are signed display formatting: -16 through -10 identify x16 through x22.

The final result trace may show -18014048 after x22 has already captured -18014128. This occurs because the updated x22 is read back and the combinational adder adds 80 again; with rw disabled, that later result is not stored. The accompanying RARS register screenshot provides the final stored-register reference, rather than the final combinational result.

The ports in mem.vhd are as follows:

DATA_WIDTH : This generic specifies the number of bits in each memory word and determines the widths of the data input and q output. Its default value is 32 bits, although the byte-write logic in this implementation is specifically written for four 8-bit bytes.

ADDR_WIDTH : This generic specifies the width of the memory address input and determines the number of available memory locations. The default value of 10 provides 1024 words locations.

BYTE_WIDTH : This generic has a default value of 8, representing the intended number of bits per byte.

clk : This signal supplies the clock for memory writes. On each rising edge, the selected bytes are written if both the write-enable signal and their corresponding byte-enable bits are asserted.

addr : This input selects the memory word to read or write. With the default 10-bit width, it selects word locations 0 through 1,023; these are word indices rather than byte addresses.

data : This input carries the value to be written into the selected memory location. Only the bytes enabled by be are stored on a rising clock edge when we is high.

be : This four-bit byte-enable signal independently controls writing to each byte of the 32-bit word. Bit 0 controls bits 7–0, while bit 3 controls bits 31–24; setting be to "1111" enables writing the entire word.

we : This signal enables memory writes when set to '1'. When it is '0', memory contents remain unchanged, but reading is still available.

q : This output provides the word stored at the location selected by addr. Reading is asynchronous, so the output responds to changes in the address or selected memory contents without waiting for a clock edge.

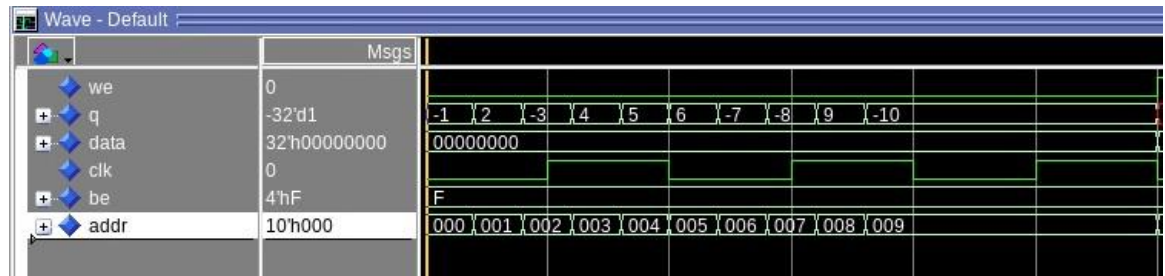
[Part 5 (c)] **Waveforms and hex file.**



Memory overview annotation

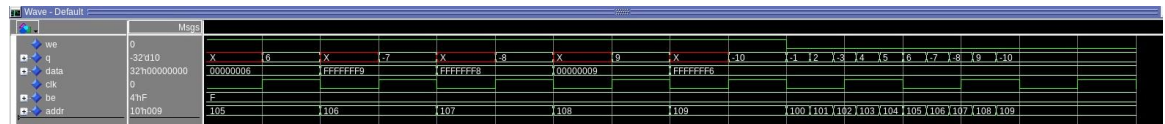
The waveform contains three phases: read the ten initialized words, copy them to RAM word addresses 0x100 through 0x109, and read the copied words back. addr selects a RAM word, data supplies write data, q is the asynchronous read output, we enables a write on a rising clk edge, and be = F enables all four bytes.

During a store, q can initially show X because the destination has not yet been written. After the rising edge, q displays the stored word. These temporary unknowns before writing do not indicate a failed copy.



Initial memory read annotation

With we = 0, addr steps from 000 through 009 and q returns -1, 2, -3, 4, 5, 6, -7, -8, 9, and -10. Reading follows the address without waiting for a clock edge. The q row is displayed as signed decimal; addr and data are hexadecimal.



Copied memory readback annotation

The last writes place 6, -7, -8, 9, and -10 into word addresses 105 through 109. After we becomes 0, addr revisits 100 through 109 and q returns the complete original sequence: -1, 2, -3, 4, 5, 6, -7, -8, 9, -10. This readback verifies that the ten words were copied correctly. These addresses are native RAM word indices, not processor byte addresses.

```
@000
FFFFFFF
00000002
FFFFFFFD
00000004
00000005
00000006
FFFFFFF9
FFFFFFF8
00000009
FFFFFFF6
```

Hex initialization file annotation

The @000 marker starts initialization at RAM word index zero. Each following eight-digit hexadecimal value is one 32-bit word; for example, FFFFFFFF represents -1 and FFFFFFF6 represents -10. The ten entries match the initial read sequence shown above.

[Part 6 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended? What are the RISC-V instructions that require some value to be appended with zero?

Sign extension is required for the 12-bit immediate of addi, slti, sltiu, andi, ori, and xori, as well as load/store address offsets and jalr. Branch and jal offsets are also sign-extended. Additionally, lb and lh sign-extend the loaded 8-bit and 16-bit data, respectively.

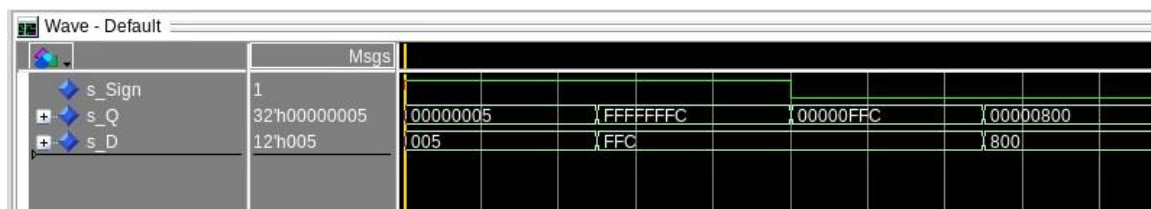
Zero extension is required for the data loaded by lbu and lhu, which fill the upper register bits with zeros. Their address offsets are still sign-extended.

Zero appending is required for lui and auipc, which place 12 zeros below their 20-bit immediate. Branch instructions and jal append one low-order zero to their reconstructed immediate bits, forming offsets in multiples of two bytes

[Part 6 (b)] what are the different “extender” components that would be required by a RISC-V processor implementation (i.e., number of bits of input and sign vs zero extension vs zero appending)?

Input width	Operation	Output width	Use
12 bits	Sign extension	32 bits	jalr
8 bits	Sign or zero extension	32 bits	lb / lbu
16 bits	Sign or zero extension	32 bits	lh / lhu
20 bits	Append 12 zeroes	32 bits	lui, auipc
12 reconstructed bits	Append one zero then sign extended	32 bits	Branch offsets
20 reconstructed bits	Append one zero then sign extended	32 bits	jal offsets

[Part 6 (d)] Waveform.



Extender waveform annotation

s_D is the 12-bit input, s_Sign selects sign extension (1) or zero extension (0), and s_Q is the 32-bit output. With s_Sign = 1, 005 becomes 00000005 and FFC becomes FFFFFFFC (-4). With s_Sign = 0, the same FFC becomes 00000FFC, and 800 becomes 00000800. The sign-controlled upper bits change while the original lower 12 bits are preserved; this circuit requires no clock.

[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

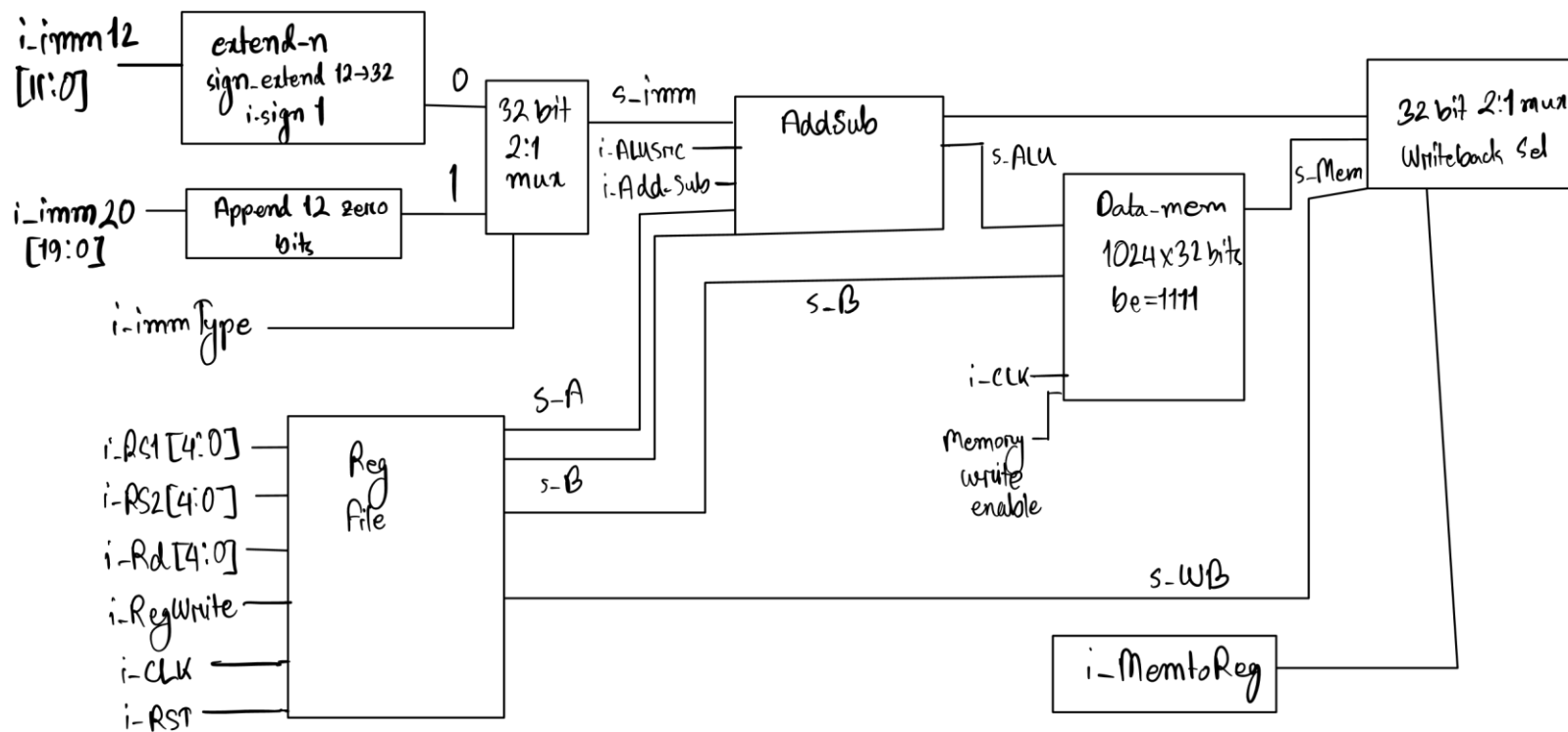
The control signals that will be needed are as follows:

MemWrite : This enables storing a register value into memory and controls the memory's (mem.vhd) we input.

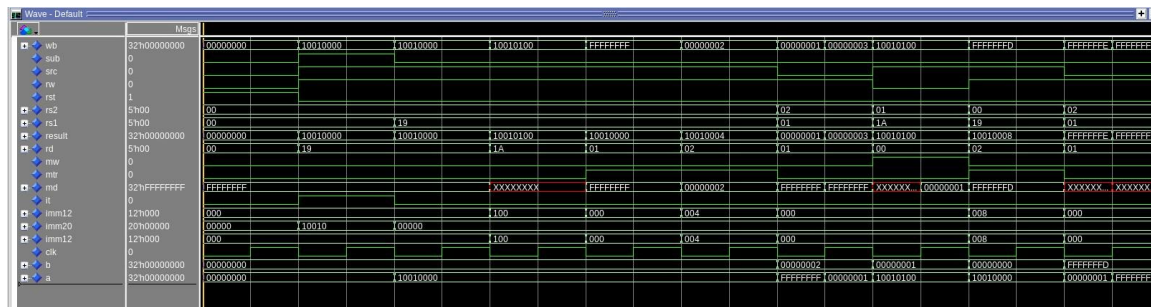
MemToReg : This selects either the ALU result or the memory output for register writeback. It controls a mux whose memory input comes from q.

ImmType : This selects the sign-extended 12-bit immediate or the 20-bit immediate width 12 appended zeroes. It controls the immediate selection before the ALU.

[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



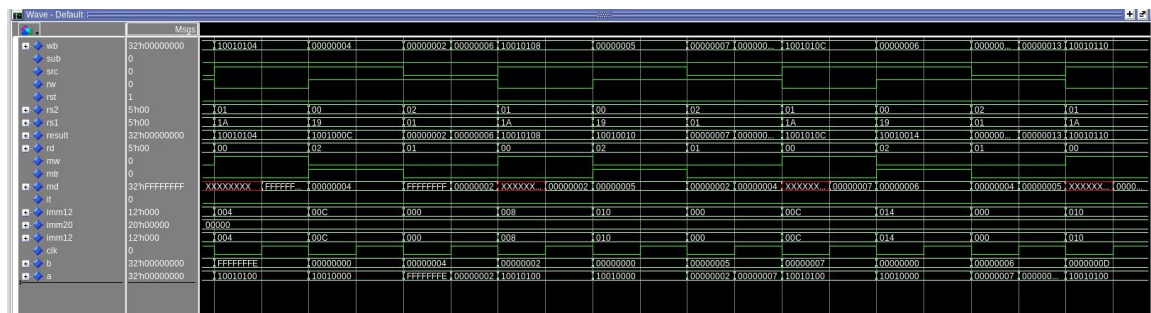
[Part 7 (c)] Waveform.



Datapath waveform 1 Setup and first operations

Signal key: rs1 and rs2 select the registers read as a and b; rd selects the write destination. result is the ALU result, md is memory read data, and wb is selected writeback data. rw and mw enable register and memory writes. src selects the immediate, sub controls the ALU operation, mtr selects memory writeback, and it selects the upper immediate. All displayed data and register indices in these three waveforms are hexadecimal.

The initial LUI and ADDI operations establish $x_{25} = 10010000$ and $x_{26} = 10010100$. Loads from 10010000 and 10010004 return FFFFFFFF (-1) and 00000002. Adding them writes 00000001 into x1, and the first store writes that value to 10010100. The next load returns FFFFFFFD (-3), producing the next accumulated result FFFFFFFE (-2).

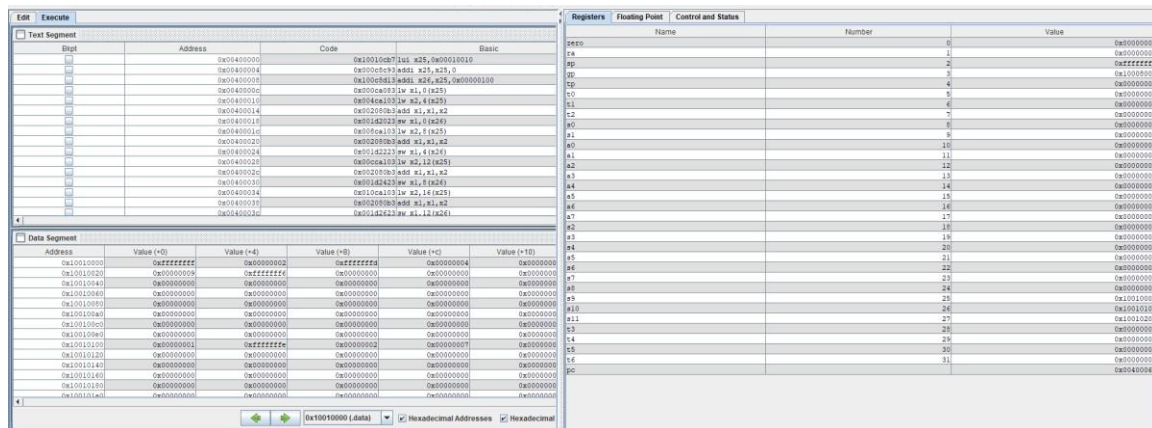


Datapath waveform 2 Accumulation and stores

The next store writes FFFFFFFE to 10010104. Loads of 4, 5, and 6 are added to the running sum, producing 2, 7, and 13. These values are stored at 10010108, 1001010C, and 10010110. For each store, result is the address, b is the word being written, mw = 1, and rw = 0. For each load, mtr = 1 routes md into wb.

Because x1 is both a source and destination for the additions, result can change again after the write edge. For example, after x1 captures 13, the adder can show $13 + 6 = 19$ (hexadecimal 13). The stored result is the value at the rising edge, not this later combinational value.

[Part 7 (d)] Compare the QuestaSim waveform and memory results with RARS.



Comparison of QuestaSim and RARS

The QuestaSim execution agrees with the visible RARS results for the accumulated values and final address registers. Both show x1 = 00000006, x2 = FFFFFFF9 (-7), x25 = 10010000, x26 = 10010100, and x27 = 10010200. RARS labels x1 as ra, x2 as sp, and x25 through x27 as s9 through s11.

The first four stored words can be directly compared with the RARS data segment screenshot. The fifth word is clipped at the right edge of the screenshot, and address 100101FC is below the displayed range; their Questa values are therefore compared with the program calculation rather than claimed as directly visible RARS matches.

Byte address	Questa stored value	RARS screenshot	Comparison
10010100	00000001 (1)	00000001	Match
10010104	FFFFFFFE (-2)	FFFFFFFE	Match
10010108	00000002 (2)	00000002	Match
1001010C	00000007 (7)	00000007	Match
10010110	0000000D (13)	Final digits clipped	Matches 7 + 6
100101FC	00000006 (6)	Outside visible range	Matches 13 - 7

RARS displays processor byte addresses. The lab RAM uses result(11 downto 2), so these locations correspond to RAM word indices 040, 041, 042, 043, 044, and 07F. The waveform also shows temporary unknown values and combinational changes that do not appear in the final RARS state; only values captured on enabled rising clock edges determine the stored results.